Ders 11 Genişletilmiş Notu: Bağlı Liste ile Kuyruk, Ağaç Terminolojisi ve İkili Ağaçta Gezinme

Ders 11 Genişletilmiş Notu: Bağlı Liste ile Kuyruk, Ağaç Terminolojisi ve İkili Ağaçta Gezinme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yığın ve Kuyruk: Hatırlatma
- 2. Dizi ile Kuyruk
- 3. Dizi mi, Bağlı Liste mi?
- 4. Kuyruk Üzerindeki İşlemler
- 5. Bağlı Liste ile Kuyruk
- 6. Ağaç Veri Yapısı ve Kullanım Alanları
- 7. Ağaç Terminolojisi
- 8. Ödev Sorusu Üzerine: Harf ve Rakamlarla Yığın
- 9. Ağacın Bellekte Gösterimi
- 10. İkili Ağaç
- 11. Ağaçta Gezinme
- 12. Preorder Fonksiyonunu Yazmak

1. Yığın ve Kuyruk: Hatırlatma

Hoca derse önceki dersin konularını hatırlatarak başladı. Yığın (stack) veri yapısında hedef, son işlem yapılan elemana ilk erişmektir. Bunu ekranda bir metin yazıp undo yaparak gösterdi: undo en son yazılanı siler, redo ise en son silineni geri getirir. Hocanın anlatımına göre bu iş aslında iki ayrı yer kullanır: birinden alınan diğerine konur.

Buradaki hedef, son yazılanı silmektir. Aradaki bir harfi silmek isterseniz bunu zaten kendiniz yapmak zorundasınız. Son yaptığınız işi geri almak ya da son yaptığınız şeye ilk erişmek istediğinizde yığın kullanılır. Hocanın hatırlattığı iki örnek daha vardır:

- **Web sayfası gezintisi:** Linklere tıklaya tıklaya ilerlersiniz. Geri geldiğinizde önce en son uğradığınız yere, sonra ondan öncekine dönersiniz.
- **Fonksiyon çağrıları (function call):** Program, en son hangi noktadan yola çıktıysa oraya geri döner.

Kuyruk (queue) benzer bir yapıdır, ama yığın son giren ilk çıkar (Last In First Out, LIFO) mantığıyla çalışırken kuyruk ilk giren ilk çıkar (First In First Out, FIFO) mantığıyla çalışır. İlk gelene ilk hizmet verilir. Bu, günlük hayattan alışık olduğumuz bir yapıdır: markette kuyruğun başındaki kişiye hizmet verilir, yeni gelen kişi kuyruğun sonuna eklenir. Bu yüzden kuyrukta iki işaretçi (pointer) vardır:

- `front`: kuyruğun başını gösterir.
- `rear`: kuyruğun sonunu gösterir.

2. Dizi ile Kuyruk

2.1 Kaydırmalı Kuyruk

İlk yöntem şudur: Bir dizi kullanılır ve baştaki elemanın işi bitince bütün dizi bir sola kaydırılır. Kaydırmanın sebebi, yerin sınırlı olmasıdır. Kimseyi yerinden kıpırdatmazsanız, B bitti, C bitti, herkes gitti ama bir noktada arkada boş yer kalmaz.

Hocanın örneğinde dizinin 0-4 gözlerinde A, B, C, D, E vardır. `front` A'yı, `rear` 4. gözdeki E'yi gösterir. A'nın işi bitince:

- B, C, D, E birer göz sola kayar. Bu 4 kaydırmadır (ara adım eklendi).
- B 0. göze yerleşir. Artık `front` B'dir.
- `rear` hâlâ E'dir, ama E 4. gözden 3. göze gelmiştir.

Bu yöntemde `front` her zaman 0. gözdür, çünkü kuyruğun başı hep dizinin ilk adresidir.

Bu yöntemin verimli olup olmadığı önceki derste konuşulmuştu. Dizinin tamamı doluysa, bir kişi çıktığına göre $n-1$ eleman kalmıştır ve bu $n-1$ elemanın hepsi öne doğru hareket ettirilmelidir. Çok hareketli bir market kuyruğunu simüle ettiğinizi düşünün: sürekli $n-1$'lik bir iş yapıyor olursunuz.

2.2 Dairesel Kuyruk

Elemanları sürekli hareket ettirmek masraflı olduğuna göre, elemanları yerinde bırakıp yalnızca işaretçileri hareket ettirmek daha verimli bir çözümdür. Dairesel kuyruk (circular queue) bu fikirdir. Yine dizi kullanılır, ama A'nın işi bitince yalnızca `front` B'ye alınır.

Örnekle ilerleyelim (dizi 0-5 gözlüdür, `front` başta A'dır, `rear` 4. gözdeki E'dir):

Adım	`front`	`rear`	0	1	2	3	4	5
Başlangıç	A (0)	E (4)	A	B	C	D	E	boş
A çıktı	B (1)	E (4)	boş	B	C	D	E	boş
F geldi	B (1)	F (5)	boş	B	C	D	E	F
G geldi	B (1)	G (0)	G	B	C	D	E	F

G, A'nın eski yerine yerleşir. `front` B olduğu için 0. göz artık boştur ve G rahatça oraya konabilir. Dairesel olması sayesinde bütün elemanları en başa kaydırmak yerine, yalnızca başı ve sonu gösteren işaretçileri hareket ettirerek problemi daha verimli çözmüş olduk.

2.3 `rear` Neyi Göstermeli?

Hoca burada kafaların karışmaması için bir açıklama yaptı. Slaytta `rear` bir yerde ilk boş gözü gösteriyordu, ama böyle olmak zorunda değildir. `rear` ister son dolu gözü, ister yeni elemanın yerleşeceği ilk boş gözü göstersin; bu tamamen sizin algoritmanıza bağlıdır. Yukarıdaki örnekte ilk boş gözü göstermek isterseniz `rear` 5 olur, son dolu gözü göstermek isterseniz 4 olur.

Dairesel kuyruğu gerçekleştiren algoritmayı yazmak öğrencilere bırakıldı: normal dizi kullanarak uygun algoritmayı kendiniz tasarlayın.

> **Ek bilgi (derste anlatılmadı):** Dairesel hareket mod işlemiyle yazılabilir: `rear = (rear + 1) % N`, burada `N` dizinin göz sayısıdır. Yukarıdaki tabloda $N = 6$ iken `rear` 5'ten $(5+1) \bmod 6 = 0$ değerine geçer, G bu yüzden 0. göze yerleşir. `front` da aynı biçimde ilerler. Dolu ve boş durumlarını ayırt etmek için eleman sayısını tutmak ya da bir gözü boş bırakmak gibi bir kural seçmek gerekir, çünkü iki durumda da işaretçilerin konumu aynı görünebilir.

3. Dizi mi, Bağlı Liste mi?

Soru: Kuyruğu bağlı listeyle (linked list) de yapabiliriz, değil mi? **Cevap:** Evet. Ama kuyruğun boyutu tahmin edilebiliyorsa doğrusu dizi kullanmaktır.

Dinamik bellek ayırma (dynamic allocation) ve bağlı liste mantığı şu durumda gerekir: Sürekli yeni elemanlar ekleniyor ve çıkıyor, kaç eleman olacağını kestiremiyorsunuz. Böyle bir durumda çok büyük bir yer tanımlamak verimli olmaz. Hoca iki durumu karşılaştırdı:

- **Market simülasyonu:** Bir kuyrukta bir anda en fazla 50 müşteri olabileceğini tahmin ediyorsanız 50'lik bir dizi tanımlar ve onu kullanırsınız.
- **Sürekli hareketli bir platform:** Eleman sayısı bazen 3, bazen 300 oluyor ve hiç kestiremiyorsanız bağlı liste kullanırsınız.

Bağlı listenin de dezavantajı vardır. Ek olarak adres değişkeni tutar: 500 elemanlık yer tanımladığınızda 500 de adres için yer tanımlamış olursunuz. Ayrıca her eklemede bellek ayırma (memory allocation) gibi ek işleri vardır. Bu yüzden bağlı listeyi gerçekten ihtiyaç varsa kullanın: hareket çok fazlaysa ve kaç eleman ekleneceğini, kaç eleman çıkacağını bilmiyorsanız. Aksi hâlde dairesel kuyrukta çok masraf yoktur; yalnızca `front` ve `rear` hareket eder.

4. Kuyruk Üzerindeki İşlemler

Kuyrukta iki temel işlem vardır:

- `enQueue`: kuyruğa yeni gelen elemanı yerleştirir.
- `deQueue`: kuyruğun başındaki kişiye hizmet verir, yani baştaki veriyi kuyruktan çıkarır.

Bu işlemlerden önce iki kontrol yapılır:

- `isFull`: Dizi statik bir yapı olduğu için bir noktada dolabilir. `enQueue` öncesinde dolup dolmadığı kontrol edilir. Dolduysa kuyruğa yeni eleman alınamaz.
- `isEmpty`: `deQueue` öncesinde kuyruğun boş olup olmadığı kontrol edilir.

Boşluk kontrolü yalnızca hata yakalamak için değildir. Evet, sistem yanlış bir kontrol yüzünden boş kuyruktan eleman çekmek isteyebilir. Ama tasarımınız zaten "kuyruk tamamen boşalana kadar hizmet et" biçiminde de olabilir. O zaman ortada hata yoktur, boşalana kadar hizmet edersiniz ve her `deQueue` öncesinde `isEmpty` ile bakarsınız.

Yığında olduğu gibi burada da bir `peek` fonksiyonu vardır. Kimseyi yerinden oynatmadan kuyruğun başında kim olduğunu verir. Kime hizmet edileceğini görmek isteyip o anda ona hizmet etmeyecekseniz `peek` ile yalnızca sorgulama yaparsınız.

5. Bağlı Liste ile Kuyruk

5.1 Mantık

Çok sirkülasyon varsa ve eleman sayısını kestiremiyorsanız bağlı liste kullanılır. Durum şöyledir: `front` A'dır, A B'yi gösterir, B C'yi gösterir, C kimseyi göstermez (`NULL`) ve `rear` C'dir.

```
front                  rear
  |                      |
  A  ->  B  ->  C  ->  NULL
```

- **`enQueue` ile D eklenirse:** C artık `NULL`'ı değil, yeni gelen D'yi gösterir. D yeni `rear` olur ve `NULL`'ı gösterir.
- **`deQueue` yapılırsa:** A kuyruktan çıkar, yeni `front` B olur.

5.2 Yapı Tanımları

Kuyruktaki her eleman iki parçadan oluşur: `key` değişkeni ve bir sonraki elemanın adresini tutan `next` işaretçisi. `key` yerine kuyrukta ne saklamak istiyorsanız o olabilir: ad, soyad, adres. Kendisi de bir `struct` olabilir. Hoca basitlik için tek bir `int` kullandı.

Hoca sonra şunu sordu: Kuyruğa erişebilmek için neyi bilmemiz gerekir? Öğrencilerin de söylediği gibi kuyruğun başını (bir öğrenci buna head dedi, burada adı `front`) ve kuyruğun sonunu. Demek ki bağlı listeyle kuyruk gerçekleştirmek için iki yapı gerekir: kuyruk düğümü ve iki işaretçiden oluşan kuyruk. Çıkarma yaparken `front`'a bakılır, yeni eleman `rear`'a eklenir.

```c
typedef struct QNode {
    int key;
```

```
    struct QNode *next;
} QNode;

typedef struct Queue {
    QNode *front, *rear;
} Queue;
```

Soru: `front` ve `rear`'ı `struct` olarak değil de normal bir değişken olarak tanımlayabilir miyiz? **Cevap:** Tanımlayamayız, çünkü adres gösterirler. Bir kuyruk düğümünün adresini gösterdikleri için tipleri `QNode` işaretçisidir, yani `QNode *`'dır.

> **Hocanın vurgusu:** Bu çok güzel bir soruydu, çünkü pek çok kişi bunu karıştırıyor. `front` ve `rear` bir düğümün adresini tutar. Gösterdikleri adres `QNode` tipinde olduğu için böyle tanımlanmaları gerekir.

Hoca `typedef` yazımında da öğrencilerle birlikte düzeltme yaptı.

Soru: `typedef` için altta bir ad yazmak gerekmiyor mu? **Cevap:** Haklısın, gerekiyor. Hoca kapanış parantezinden sonraki adları ekledi.

Soru: `struct` adı ile `typedef` adı aynı olabilir mi? **Cevap:** Hoca önce aynı ad olmaz diye hatırladığını söyledi, öğrenciler oluyor dedi. Yukarıdaki kod gerçekten de `gcc -Wall` ile uyarısız derlenir.

Bir öğrenci de `struct` adının içeride kullanılmadığı sürece gerekmediğini belirtti. `QNode`'da içeride kullanıldığı için gereklidir.

5.3 Kuyruğu Oluşturmak: `createQueue`

```
Queue *createQueue()
{
    Queue *q = (Queue *)malloc(sizeof(Queue));
    q->front = q->rear = NULL;
    return q;
}
```

Burada kuyruk için yer ayrılır, `front` ve `rear`'a `NULL` atanır. Kuyruk şu anda boştur ve boş olduğunu göstermek için başına ve sonuna `NULL` yazmak gerekir. Hoca bunun aslında bir başlatma (initialization) olduğunu söyledi: başlangıçta boş bir kuyruk tanımladık.

> **Ders sonrası not:** Hoca burada "bir kuyruk işaretçisi için yer ayırdım" dedi. Kodda ayrılan yer `sizeof(Queue)` kadardır, yani `front` ve `rear` alanlarını içeren `Queue` yapısı için ayrılır. `q` ise bu yapının adresini tutan işaretçidir.

Soru: `createQueue` değer döndürüyor ama `return` yok. **Cevap:** Haklısın. Bu uyarıyla `return q;` satırı eklendi.

5.4 Kuyruğu Global mi Tanımlamalı?

Kuyruğu global tanımlamak mümkündür. Bu küçük programda yalnızca kuyrukla işlem yapıldığı için global tanımlamakta sıkıntı yoktur. Ama pek çok fonksiyondan oluşan büyük bir programda kuyruk her yerde kullanılmıyorsa global tanımlamak hoş değildir. Hocanın iki gerekçesi vardır:

- Bir yerde bir şey bozulduğunda takip etmek zorlaşır.
- Global değişkenler program çalıştığı sürece bellekte yer tutar.

Bu yüzden hoca kuyruğu parametre olarak aktardı ve buna dikkat edilmesini istedi.

5.5 Eleman Eklemek: `enQueue`

```
void enQueue(Queue *q, int key)
{
    QNode *tmp = newNode(key);
    if (isEmpty(q))
```

```
    {
        q->front = q->rear = tmp;
    }
    else
    {
        q->rear->next = tmp;
        q->rear = tmp;
    }
}
```

Soru: Burada bellek ayırmamız (allocation) gerekmez mi? **Cevap:** `newNode` fonksiyonu ayırıyor, onu çağırıyoruz.

İki durum vardır.

Kuyruk boşsa: Yeni düğüm hem başlangıç hem son düğümdür, yani `front` da `rear` da `tmp`'yi gösterir. Hocanın çiziminde `tmp`'de 9 vardır ve `next` alanı `NULL`'dır. İş bu kadardır. `if` bloğunun sonunda `return` deseniz de olur, demeseniz de olur, çünkü zaten `else` vardır.

Kuyruk boş değilse: Gelen eleman kuyruğun sonuna eklenir. Hocanın çiziminde bir `front`, bir `rear` ve değeri 3 olan yeni bir `tmp` vardır. Sıra önemlidir:

1. `rear`'ın `next` alanı artık `NULL`'ı değil `tmp`'yi göstermelidir: `q->rear->next = tmp;`
2. Yeni kuyruk sonu bu elemanın adresi olmalıdır: `q->rear = tmp;`

Kuyruk boşken başı da sonu da `tmp` gösterir. Kuyrukta eleman varken yalnızca sonu değişir.

Soru: `q->rear = tmp` yerine başka bir şey mi yazmamız gerekiyor? **Cevap:** Öğrenci sorusunu kendi kendine yanıtladı, kodda `q->rear = tmp;` kaldı.

5.6 `isEmpty`

`enQueue` içinde iki fonksiyon çağrılıyor: `newNode` ve `isEmpty`. `isEmpty` derste yazılmadı, öğrencilere bırakıldı. Hocanın tarifi şudur: hem `q->front` hem `q->rear` `NULL` ise kuyruk boştur.

> **Hocanın vurgusu:** Lütfen kendiniz yazın. Bir şeylerle oynamazsanız, bir şeyler yapmazsanız öğrenemezsiniz.

5.7 Eleman Çıkarmak: `deQueue`

`deQueue` kuyruğun başındaki kişiyi kuyruktan çıkarır. Burada da `isEmpty` kullanılır, ama sebebi farklıdır. `enQueue`'da boşluğa bakma sebebi, boş kuyrukta `front` ve `rear`'ı yeni elemana bağlamaktı. Burada ise kuyruk boşsa yapılacak iş olmadığı için bakılır. Hoca bu durumda yalnızca mesaj yazdı; ne yapılacağı algoritmanın ihtiyacına bağlıdır.

Kod derste öğrencilerin uyarılarıyla birkaç adımda şekillendi. Son hâli şöyledir:

```
int deQueue(Queue *q)
{
    int key;
    QNode *tmp = q->front;
    if (isEmpty(q))
    {
        printf("Queue is empty\n");
        return -1;
    }
    else
    {
        key = q->front->key;
        q->front = q->front->next;
        free(tmp);
        if (q->front == NULL)
            q->rear = NULL;
        return key;
```

```
    }
}
```

Adım adım, iki elemanla. A 1000 numaralı adreste, B 1500 numaralı adreste olsun. A'nın `next` alanı 1500'dür, B'nin `next` alanı `NULL`'dır. `front` 1000'i gösterir, `rear` 1500'ü gösterir.

İşlem	Sonuç
`tmp = q->front`	`tmp` 1000'i gösterir
`key = q->front->key`	A'nın değeri alınır, sonra geri döndürülecek
`q->front = q->front->next`	`front` 1500'ü, yani B'yi gösterir
`free(tmp)`	1000 numaralı adres serbest bırakılır
`q->front == NULL` mı?	Hayır, `rear` ile ilgili iş yok

Hocanın dediği gibi A'nın işi bitti: hizmet ettiniz, yazıcıda çıktısını bastınız, ne yaptıysanız yaptınız. Kuyruğun başı artık A'nın `next`'i olmalıdır.

Soru: Bu işlemden önce `front`'u `free` etmemiz gerekmiyor mu? **Cevap:** Hoca "çok güzel bir soru, bu işlemden önce `free` edersen ne olur?" diye sordu. Öğrenci, önce `front`'u bir `tmp`'ye atayıp, `front` B'ye geçtikten sonra `tmp`'yi `free` etmeyi düşündüğünü söyledi. Kodda da böyle yapıldı.

Soru: `deQueue`'nun `key` döndürmesi gerekmez mi? **Cevap:** Evet. Hoca başta yalnızca çıkarmayı yazmıştı. Kuyruktan çıkarmak sembolik bir işlem olduğu için, hizmet edildiğini çıkan değeri geri döndürerek gösteriyoruz.

Neden `free`? Hoca bunu şöyle anlattı: `free` yapmazsanız bir süre sonra bellek şişer. Bunlar ayrılmış ama kullanılmayan, boş olmasına rağmen kullanamadığınız yerlerdir. Üstelik program hata mesajı vermediği için hata yaptığınızı anlamazsınız, ama program verimli çalışmaz.

`tmp`'nin tipi. Hoca `tmp`'yi önce yanlışlıkla `Queue` tipinde yazdı. Öğrencilerin uyarısıyla düzeltti.

Soru: `tmp`, `QNode` tipinde bir işaretçi değil mi? `q->front`'a bir düğüm atıyoruz, `tmp` de `QNode` olmalı. **Cevap:** Haklısınız. `front` bir kuyruk düğümünün adresini gösterdiği için tipi `QNode *`'dır. `tmp`'nin de `front` gibi bir adres göstermesi isteniyor, bu yüzden o da `QNode *` olur. Sonra o adresi serbest bırakmak için `tmp` kullanılır.

Hoca, kendisi fark edemeyebileceği için anlatırken yaptığı hataları öğrencilerin düzeltmesini istedi.

Hata dönüş değeri. Kuyruk boşsa `-1` döndürülür. Bu, `key` değerinin negatif olmayacağı düşünülerek seçilmiştir. Algoritmaya göre farklı bir değer de dönebilir. Hocanın başka bir örneği: kuyruktan çıkarabildiyseniz 1, çıkaramadıysanız -1 döndürmek.

5.8 Tek Eleman Durumu

Hoca kodu burada bitirmedi ve "ne kaldı geriye?" diye sordu. Bir öğrenci `rear` kontrolünü önerdi.

Soru: Orada bir kontrol yapmamız gerekmez mi? `front` 1500 yerine `NULL`'ı gösterebilirdi, yani kuyrukta tek eleman olabilirdi. **Cevap:** Evet, asıl sıkıntı budur.

Birden fazla eleman varken sıkıntı yoktur. İki elemanlı durumda da, A, B, C durumunda da `front` değişir, `rear` olduğu yerde sonda durur.

Tek elemanlı durumda ise `front` da `rear` da aynı düğümü gösterir, çünkü boş kuyruğa eklerken ikisini de `tmp`'ye eşitlemiştik. A'nın değeri 9 olsun ve hocanın "yavaş çekim" gösterdiği adımları izleyelim:

1. `key = 9` alınır.
2. `q->front = q->front->next` çalışır. `next` `NULL` olduğu için `front` `NULL` olur, çünkü kuyrukta başka eleman yoktur.
3. `free(tmp)` ile düğüm serbest bırakılır.
4. `rear` ise hâlâ serbest bırakılan yeri gösterir. Bu olmaz.

Bu yüzden şu kontrol gerekir: `front` `NULL` olduysa, yani kuyruk boşaldıysa `rear` da `NULL` yapılır. Kuyrukta başka eleman varsa `rear` ile ilgili işlem gerekmez.

> **Ek bilgi (derste anlatılmadı):** Bu satır yazılmazsa `isEmpty` yanlış sonuç verir, çünkü hocanın tarifine göre iki işaretçinin de `NULL` olması gerekir. Satır silinerek derlenen sürümde `isEmpty` kuyruk boş olduğu hâlde 0 döndürür. Sonraki `deQueue` `NULL` adresinden okumaya çalışır ve program bozulur; AddressSanitizer bunu bellek hatası olarak gösterdi. Sonraki `enQueue` ise serbest bırakılmış düğümün `next` alanına yazar.

5.9 Yeni Düğüm: `newNode`

`enQueue`'da yeni eleman için önce yer ayrılır, sonra gelen değer oraya yerleştirilir. Dönüş değeri, ayrılan yerin adresidir. Bu yüzden fonksiyon `QNode *` döndürür ve parametresi yeni düğümün değeridir.

```
QNode *newNode(int key)
{
    QNode *tmp = (QNode *)malloc(sizeof(QNode));
    tmp->key = key;
    tmp->next = NULL;
    return tmp;
}
```

`next` `NULL` yapılır, çünkü yeni eleman kuyruğun en sonuna yerleşecek ve kimseyi göstermeyecektir. Hocanın çiziminde `front` A'yı, A B'yi, B `NULL`'ı gösterir. Yeni eleman C'dir. C `NULL`'ı gösterir, birazdan B de C'yi gösterecektir. Fonksiyon adresi döndürür, `enQueue` da bu adresi kullanarak elemanı kuyruğun sonuna ekler.

> **Hocanın vurgusu:** Bellek ayırma yaptığınız her yerde yer tutabildiniz mi diye mutlaka kontrol edin. Tutulamadıysa hata mesajı verin. Yer ayrılamadığı hâlde ayrılmış gibi davranmaya devam ederseniz çok sıkıntı olur. Hoca bu kontrolü derste yalnızca vakit kaybetmemek için yazmadı.

Hoca bu kontrolün her zaman yer kalmaması yüzünden gerekmediğini de ekledi ve yazmaya korumalı (write protected) bir yere ayırma yapmak istemeyi örnek verdi.

> **Ders sonrası not:** `malloc`'a hedef adres verilmez, yeri `malloc` seçer. Dolayısıyla yazmaya korumalı bir yeri seçmek söz konusu değildir. C standardına göre istek karşılanamazsa `malloc` `NULL` döndürür; kontrol bunun için gereklidir.

5.10 Ana Program

```
Queue *q = createQueue();
enQueue(q, 10);
enQueue(q, 20);
printf("%d\n", deQueue(q));   /* 10 */
```

Bu noktada kuyruğun durumu şudur: `front` 10'u gösterir, 10 20'yi gösterir, 20 `NULL`'ı gösterir. Hoca ekranda ne görüleceğini sordu; öğrenciler 10 dedi. Kuyruğun başında 10 vardır, bu yüzden ilk `deQueue` 10 döndürür. Bir kez daha çağrılırsa 20 görülür.

> **Ders sonrası not:** Bu bölümdeki kod parçaları, `stdio.h` ve `stdlib.h` eklenerek, `isEmpty` ve `newNode` `enQueue`'dan önce tanımlanarak `gcc -Wall` ile uyarısız derlendi ve çalıştırıldı. Bunun için `isEmpty`, hocanın tarifine göre `return q->front == NULL && q->rear == NULL;` olarak yazıldı. İki `deQueue` 10 ve 20 döndürdü. Boş kuyrukta üçüncü `deQueue` "Queue is empty" yazıp -1 döndürdü. Kuyruk boşaldıktan sonra yapılan yeni `enQueue` ve `deQueue` de doğru çalıştı. AddressSanitizer ile bellek hatası görülmedi.

6. Ağaç Veri Yapısı ve Kullanım Alanları

Kuyruk burada bitti ve yeni bir veri yapısına, ağaca (tree) geçildi. Dizi doğrusal (linear) bir veri yapısıdır: bilgilere belli bir sırayla erişilir. Ağaç ise bilgilere sıralı erişmediğimiz bir yapıdır.

Ağaç bir hiyerarşiyi göstermek için kullanılır. Gerçek dünyadaki ağaç simüle edilerek oluşturulmuştur: kökü, yaprakları ve arada yer alan bilgileri vardır. Hocanın saydığı kullanım alanları şunlardır:

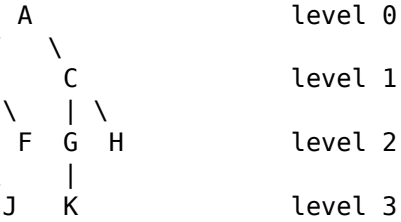
- **Aile ağacı:** En basit bildiğimiz örnektir. Büyükbaba ve büyükanne tepede (kökte), çocukları ve çocuklarının çocukları sırayla aşağıdadır.
- **Bilgisayardaki dizin (directory) yapısı:** Bilgisayarımızın organizasyonu da bir ağaçtır (aşağıda).
- **Karar ağaçları (decision tree):** En çok kullanılan ağaç türlerinden biridir. Bir A durumundasınız; A 50'den büyükse B'ye, 50'den küçük veya eşitse C'ye gidersiniz; bir değer pozitifse D'ye, negatifse E'ye gidersiniz. Hiyerarşiye göre ağaç üzerinde ilerlersiniz. Yapay zekâda çok kullanılan bir öğrenme yöntemidir: bilinmeyen bir örneğin sınıfını bulmak için kullanılabilir.
- **Satranç gibi oyunlar:** Yapılan hamleye göre ilerlenir. "Bu yapılırsa buradan, şu yapılırsa şuradan" biçiminde oyunun nasıl açılacağı ağaç yapısıyla belirlenir.
- **Organizasyon şemaları:** Bir öğrencinin önerdiği örnektir. Şirketin genel müdürü, departmanları ve departmanların altındaki birimler bir ağaçtır.

Dizin örneğinin yapısı şöyledir:

```
root
├── Dersler
│   ├── Data Structure
│   │   ├── Notlar
│   │   └── Ödevler
│   └── Software Engineering
└── Oyunlar
```

7. Ağaç Terminolojisi

Hoca bu terimlerin uzun süre karşımıza çıkacağını, çünkü ağacın çok farklı uygulamalarda kullanıldığını söyledi. Terminoloji şu örnek ağaç üzerinden anlatıldı. Ağacı, gerçek dünyadaki ağacın ters dönmüş hâli gibi düşünün.



7.1 Düğüm Türleri

Ağaçta 11 düğüm (node) vardır. Her düğüm tek başına anlam ifade eder ve bulunduğu yere göre adlandırılır.

- **Kök (root):** Başlangıç düğümüdür ve her zaman tektir. Öncesinde hiçbir şey yoktur. Burada A'dır.
- **Yaprak (leaf):** Çocuğu olmayan düğümdür. Burada D, F, H, I, J, K.
- **İç düğüm (internal node):** En az bir çocuğu olan düğümdür. Burada A, B, C, E, G. Kök de bu tanıma göre iç düğümdür.
- **Ebeveyn düğüm (parent node):** Kendisine bağlı alt düğüm olan düğümdür. A, B ve C'nin ebeveynidir. D ise yaprak olduğu için ebeveyn değildir.
- **Çocuk düğüm (child node):** Kendisinden önce gelen düğüm olan düğümdür. B ve C A'nın, D, E, F B'nin, I ve J E'nin çocuklarıdır.
- **Kardeş düğüm (sibling node):** Aynı ebeveyne ait düğümlerdir. B ve C kardeştir; D, E, F de kardeştir.

Soru: Çocuk düğüm için kök hariç hepsi diyebilir miyiz? **Cevap:** Evet, kök hariç hepsi.

Soru: İç düğüm ile ebeveyn düğüm tamamen aynı şey mi? **Cevap:** Evet, aynı şey. Bir tane çocuğu varsa iç düğüm diyorsunuz; bu düğümlerin hepsi aynı zamanda ebeveyndir.

> **Ders sonrası not:** Altyazıda bu cevapta iç düğümler “A, D, C, E, F, G” diye geçiyor. Doğrusu A, B, C, E, G’dir; D ve F yapraktır. Hoca iç düğümleri 01:27’de doğru saymıştı. Benzer biçimde 01:29’da kardeşler “B, E, F” diye geçiyor, doğrusu D, E, F’dir.

7.2 Derece

Düğümün derecesi (degree), o düğümün kaç çocuğu olduğudur. B’nin derecesi 3, F’nin 0, G’nin 1’dir.

> **Ders sonrası not:** Hoca başlığı “ağacın derecesi” diye açtı, ama verdiği tanım tek bir düğümün derecesidir. Ağacın derecesi denildiğinde düğüm dereceleri arasındaki en büyük değer kastedilir; bu ağaçta B sayesinde 3’tür.

7.3 Seviye, Derinlik ve Yükseklik

Hoca bu kavramların ağacın bütünü hakkında bilgi verdiğini söyledi.

Seviye (level) kökten uzaklıktır: kök seviye 0’dır, sonra 1, 2, 3 diye iner. Aynı kavram için **derinlik (depth)** de kullanılır. Kökten B’ye bir adımda, K’ye üç adımda ulaşılır.

Yükseklik (height) ise düğümün yapraktan uzaklığıdır. Bu ağaçta:

Düğüm	Yükseklik	Seviye
A	3	0
B	2	1
C	2	1
E	1	2
G	1	2
D, F, H	0	2
I, J, K	0	3

Ağacın yüksekliği, kökün yüksekliğidir. Burada kökün en alt katmandaki yapraklara (I, J, K) uzaklığı 3 olduğu için ağacın yüksekliği 3’tür. Ağacın derinliği de kökten yapraklara en uzun yoldur ve yine 3’tür.

Soru: Bu yükseklik kargaşa oluşturmuyor mu? D, F, H seviye 2’de, I, J, K seviye 3’te, ama hepsinin yüksekliği 0. **Cevap:** Hayır. Genelde konuşulan hep ağacın yüksekliğidir. Bir düğümün derinliğini, kökten yola çıkınca o düğüme kaç adımda ulaşacağınızı bilmek için istersiniz: B’ye bir adımda, K’ye üç adımda, derinliği 2 olan bir düğüme iki adımda. Ağacın yüksekliğini ise bir noktadan çıkıp en uzak noktaya kaç adımda varacağınızı bilmek için istersiniz. Her bilgi algoritmanızın ihtiyacına göre kullanılır. Hepsi aynı sırada olsaydı zaten ağaç yapısının anlamı kalmazdı.

Soru: Yükseklik bilgisini düğümle beraber tutmakta fayda var mı? Örneğin C’nin yüksekliği 2’dir, bunu G ve K üzerinden giderek buluyoruz, H üzerinden bulamıyoruz. Bütün çocukları kontrol etmek yerine saklamak iyi olmaz mı? **Cevap:** Algoritmana bağlı. Normalde ağacın `struct`’ında her düğüm için yükseklik saklamayız. Ama sürekli kullanman gerekiyorsa haklısın; o zaman bir kez hesaplar, bir yerde tutarsın.

7.4 Kenar ve Yol

Kenar (edge), iki düğümü birbirine bağlayan bağlantıdır. A ile B’yi bağlayan bir kenardır. Kenar, düğümler arasındaki geçişi sağlayan anlamlı bir bilgi de taşıyabilir:

- Bir karar ağacında “renk” düğümünden kırmızı, yeşil, pembe kenarları çıkar. Kırmızıysa “yuvarlak mı?” diye sorulur, oradan da evet ve hayır kenarları çıkar.
- İstanbul ve Ankara iki düğümse, aralarındaki kenar örneğin 1500 kilometre gibi bir mesafe bilgisini gösterebilir.

Yol (path), bir düğümden başka bir düğüme nasıl gidileceğini gösterir. A’dan E’ye yol A, B, E’dir.

> **Ders sonrası not:** Altyazıda A'dan E'ye yol "A, B, D, E", B'den C'ye yol "B, C" diye geçiyor. Bu örnek ağaçta D ile E kardeştir, aralarında kenar yoktur; aynı şey B ile C için de geçerlidir. Doğrusu A'dan E'ye A, B, E; B'den C'ye B, A, C'dir.

Ağaçta dikkat edilecek nokta şudur: H'den D'ye gelen bir bağlantı gibi bir şey yoktur. Her düğümün tek ebeveyni vardır. Bir düğüme ikinci bir ebeveyn bağlanması söz konusu değildir. Bu yüzden ağaçta A'dan E'ye giderken on farklı yol olamaz.

> **Hocanın vurgusu:** Terminolojiyi ezberlemeniz gerekmiyor, kullana kullana öğreniyorsunuz. En çok kullanılanlar kök ("bu ağacın kökü nedir"), yaprak ("yapraklara nasıl gideceğiz") ve yoldur ("A'dan E'ye nasıl gideceğiz").

7.5 Alt Ağaç ve Orman

Alt ağaç (subtree), bir düğümün çocuklarından itibaren oluşan ağaçtır. A'nın iki çocuğu olduğu için iki alt ağacı vardır: B'nin başlattığı ve C'nin başlattığı. C'nin aşağısı da C'nin çocuklarına göre alt ağaçlardan oluşur. Alt ağaçları kendi içinde değerlendirmek mümkündür. Örneğin A'nın sol tarafında ilerleyecekseniz yalnızca B alt ağacıyla ilgilenirsiniz, C alt ağacıyla hiç ilgilenmezsiniz.

Orman (forest), aynı amaç için kullanılan ve birbirine bağlı olmayan birden fazla ağaçtan oluşan yapıdır. Hocanın örneği: aynı apartmandaki farklı ailelerin aile ağaçları bir orman oluşturur, çünkü birbirlerine bağlı değildirler. Canlıları gösteren bir yapı ise böyle olmaz, çünkü onun tek bir tepesi vardır. Ormanda ağaçların en tepede birbirine bağlanmaması gerekir.

8. Ödev Sorusu Üzerine: Harf ve Rakamlarla Yığın

Hoca terminolojiden sonra konuyu değiştirip ödevdeki bir soruyu konuştu. Girdi `A, B, C, 2, D, E, 1` gibi harf ve rakamlardan oluşur. Harfler yığına itilir (push). Rakam görülünce o kadar eleman yığının üstünden çekilir (pop).

Girdi	İşlem	Yığın (alttan üste)
A	push	A
B	push	A, B
C	push	A, B, C
2	iki kez pop	A
D	push	A, D
E	push	A, D, E
1	bir kez pop	A, D

Hoca sorunun basit olduğunu, `push` ve `pop` fonksiyonları zaten yazıldığı için stack ile daha kolay çözüleceğini söyledi.

Soru: Soruyu stack yerine string üzerinden çözdüm, olur mu? **Cevap:** Soruda stack ile çözün denmedi, stack ile çözülebileceğini görmeniz bekleniyordu. Stack kullanmadan $O(n)$ karmaşıklıkla çözdüyseniz sıkıntı yok. $O(n)$ ile çözemediyseniz o kötüdür. Stack'siz $O(n)$ ile çözmek ise çok zordur.

Soru: Örneğin 9 sayısı gelince 9 karakter atmak için ayrı bir `for` dönüyoruz. Bu $O(n)$ karmaşıklığı ortadan kaldırır mı? **Cevap:** Algoritmayı görmeden bir şey diyemem. Ama sürekli ileri geri sarıyorsanız n'i geçer. Geçmeden çözebildiyseniz sıkıntı yok.

> **Ek bilgi (derste anlatılmadı):** Yığınla çözümde rakam için dönen `for`, karmaşıklığı $O(n)$'in üstüne çıkarmaz. Çünkü her harf en fazla bir kez itilir ve en fazla bir kez çekilir, dolayısıyla bütün `for` döngülerindeki `pop` sayısı toplamda harf sayısını aşamaz. Derste geçen girdide 5 `push` ve 3 `pop` vardır, toplam 8 işlem yapılır. Bu tür toplam maliyet hesabına amortize (amortized) analiz denir.

9. Ağacın Bellekte Gösterimi

Ağaç soyut (abstract) bir yapıdır. Bilgisayar ortamında tanımlı bir ağaç yapısı yoktur. Örneğin `int A[10]` dediğinizde art arda 10 yer tutulur; bağlı listeyi de kendiniz kurabilirsiniz. Ama yığın ve kuyrukta olduğu gibi ağaç için de hazır bir tanım yoktur, onu sizin kurmanız gerekir.

9.1 Dizi ile Gösterim

Dizi kullanmak özellikle genel ağaçlarda kolay değildir, ikili ağacı saklamak daha kolaydır. Her düğümün en fazla kaç çocuğu olabileceğini bilmeniz ve o kadar yer açmanız gerekir. Yukarıdaki ağaçta her düğümün en fazla 3 çocuğu olacağını biliyorsanız seviye seviye ilerler ve olmayan çocuklar için boş göz bırakırsınız:

İndis	0	1	2	3	4	5	6	7	8	9
Değer	A	B	C	boş	D	E	F	G	H	boş

A'nın 2 çocuğu olduğu için 3. göz boştur; oraya bir düğüm gelirse yerleşir. B'nin çocukları 4, 5, 6. gözlerdedir. C'nin çocukları 7, 8, 9. gözlerdedir (G ve H'nin yeri, ara adım eklendi). Bu yöntem hayli sıkıntılıdır: çok fazla boş göz oluşur ve en fazla çocuk sayısının hiçbir zaman aşılmayacağından emin olmanız gerekir.

9.2 Bağlı Liste ile Gösterim

Bu yüzden ağaçlarda genellikle bağlı liste yapısı kullanılır. Hoca bağlı listeleri anlatırken "çok kullanılır, iyi öğrenmeye çalışın" demesinin sebebini burada hatırlattı.

Her düğümde en fazla çocuk sayısı kadar işaretçi bulunur. Hocanın çizdiği örnekte A'nın işaretçilerinden biri B'yi, biri C'yi gösterir, biri `NULL`'dır. B'nin işaretçilerinden biri D'yi gösterir, biri `NULL`'dır, biri E'yi gösterir.

Bu yöntem silme gibi işlemlerde çok daha kolaydır. Örneğin G düğümünü silerseniz C'nin ilgili işaretçisini G'nin gösterdiği düğümü gösterecek biçimde değiştirirsiniz. Ağacı yeniden düzenlemekte güçlük çekmezsiniz. Hoca ayrıntıyı, daha çok kullanacağımız ikili ağaçta göstereceğini söyleyip burada kesti.

10. İkili Ağaç

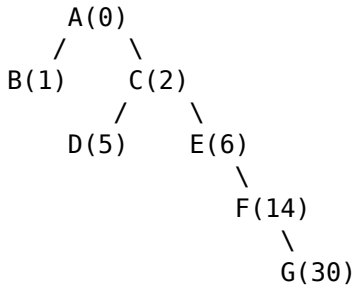
İkili ağaç (binary tree), bir düğümün en fazla iki çocuğunun olabildiği ağaçtır. Hoca bunu ayrık matematikte de görmüş olmamız gerektiğini söyledi. Yapı şöyledir: bir kök, köke bağlı bir sol alt ağaç (left subtree) ve bir sağ alt ağaç (right subtree).

10.1 Dizi ile Gösterim

Her düğümün en fazla iki çocuğu olacağı kesin olduğu için dizide yerleşim kuralı basittir:

- i. düğümün sol çocuğu $2i + 1$. gözdedir.
- i. düğümün sağ çocuğu $2i + 2$. gözdedir.

Hocanın örneği şu ağaçtır:



Yerleşim adım adım şöyle bulunur:

Düğüm	Ebeveyn	Hesap	Göz
A	kök		0
B	A, sol	$2 \cdot 0 + 1$	1
C	A, sağ	$2 \cdot 0 + 2$	2
D	C, sol	$2 \cdot 2 + 1$	5
E	C, sağ	$2 \cdot 2 + 2$	6
F	E, sağ	$2 \cdot 6 + 2$	14
G	F, sağ	$2 \cdot 14 + 2$	30

D neden 3. gözde değil de 5. gözdedir? Çünkü 3 ve 4. gözler B'nin çocuklarına aittir. B'nin çocuğu olmasa da o gözler boş kalır.

Burada da sıkıntı görülür. G'den önce 6 göz doludur, ama en az 15 göze ihtiyaç vardır. G de eklenince 7 düğüm için 31 göz gerekir ve $31 - 7 = 24$ göz boş kalır (ara adım eklendi). Boş yer problemi büyüktür.

> **Ek bilgi (derste anlatılmadı):** Yüksekliği h olan bir ikili ağaç bu yerleşimle en kötü durumda $2^{h+1} - 1$ göz ister, çünkü seviye k'de en fazla 2^k göz vardır ve toplam $\sum_{k=0}^{h} 2^k = 2^{h+1} - 1$ olur. Derste geçen ağaçta $h = 4$ olduğundan $2^5 - 1 = 31$ çıkar. Bu, G için bulunan 31 göz ile aynıdır.

10.2 Bağlı Yapı ile Gösterim

Bu yüzden ikili ağaç genellikle bağlı yapıyla gösterilir. Her düğümde değer, sol adres (left) ve sağ adres (right) vardır. Örnekte A'nın `left`'i B'yi, `right`'ı C'yi gösterir. B'nin `left`'i ve `right`'ı `NULL`'dır. C'nin `left`'i D'yi, `right`'ı E'yi gösterir.

Düğüm yapısında bir `int` değer ile düğüm tipinde iki işaretçi bulunur. Değer yine bir `struct` olabilir: kişilerin adı, soyadı, adresi, kimlik numarası gibi.

11. Ağaçta Gezinme

Ağacın tamamı üzerinde bir işlem yapmak istiyorsanız düğümleri belli bir sırayla gezmeniz (traversal) gerekir. Üç tür gezinme vardır. Hoca bunu aritmetik ifade ağacı (arithmetic expression tree) üzerinden gösterdi. Öğrenciler bunu ayrık matematikte gördüklerini söyleyince üzerinde çok durmadı. `A + B` ifadesinin ağacında kök `+`, sol çocuk `A`, sağ çocuk `B`'dir.

Gezinme	Sıra	`A + B` için	İfade gösterimi
Preorder	düğüm, sol, sağ	`+ A B`	ön ek (prefix)
Inorder	sol, düğüm, sağ	`A + B`	ara ek (infix)
Postorder	sol, sağ, düğüm	`A B +`	son ek (postfix)

11.1 `A + B * C` Örneği

Hocanın örneğinde kök `+`, sol çocuk `A`, sağ çocuk `*`'dır. `*` düğümünün çocukları `B` ve `C`'dir.



Preorder (düğüm, sol, sağ). Düğümden başlanır, `+` yazılır. Sol, `A`'dır. Sağ alt ağaca geçilir ve önce onun kökü `*` yazılır, sonra sol `B`, sağ `C`.

Inorder (sol, düğüm, sağ). Önce sol `A`, sonra düğüm `+` yazılır. Sağa gelince sağın solundan başlanır: `B`, sonra `*`, sonra `C`.

Postorder (sol, sağ, düğüm). Önce sol `A` yazılır. Sağ alt ağaca geçilir: onun solu `B`, sağı `C`, sonra kendi düğümü `*`. Sol ve sağ bittiği için en son kök `+` yazılır.

Sıra	1	2	3	4	5	Sonuç
Preorder	`+`	`A`	`*`	`B`	`C`	`+ A * B C`
Inorder	`A`	`+`	`B`	`*`	`C`	`A + B * C`
Postorder	`A`	`B`	`C`	`*`	`+`	`A B C * +`

Inorder numaralandırmasını hoca yanlış yazınca bir öğrenci düzeltti: `A` 1, `+` 2 olmalıydı.

> **Hocanın vurgusu:** Burada önemli olan şudur: sol dediğinde sol alt ağacı bir bütün olarak görmelisin ve oradaki bütün işleri bitirmelisin. Sağ dediğinde de sağ alt ağaçtaki bütün işleri bitirmelisin. Düğümle ilgili kısma sırası gelince geçersin.

12. Preorder Fonksiyonunu Yazmak

Preorder gezinmede önce düğüm işlenir, sonra sol alt ağaç, sonra sağ alt ağaç. Sol alt ağaç da kendi içinde alt ağaçlardan oluştuğu için bu problem özyinelemeli (recursive) olarak çözülür. Hoca düğümü tanımlayıp ağaçtaki bilgileri preorder sırasıyla ekrana yazan fonksiyonu yazdı:

```c
typedef struct node
{
    int value;
    struct node *left, *right;
} NODE;

void printPreorder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        printf("%d\n", treeNode->value);
        printPreorder(treeNode->left);
        printPreorder(treeNode->right);
    }
}
```

Düğüm `NULL` ise yazacak bir şey yoktur, bu yüzden iş yalnızca `NULL`'dan farklıysa yapılır. Önce düğümün değeri yazılır. Değer 9 ise 9, karakterse A yazar. Sonra fonksiyon sol çocuk için çağrılır. Orada da her seferinde `NULL`'dan farklıysa değer yazılır ve onun solu çağrılır; böylece çağrı çağrı aşağı inilerek sol taraf bitirilir. `NULL`'a gelindiğinde fonksiyondan çıkılır ve bir alt satıra, sağ çocuk çağrısına düşülür. Sol tarafın tamamı bitince sağ için aynı işlem çalışır.

Hocanın tahtadaki örneğinde A'nın `left`'i B'yi, `right`'ı E'yi gösterir. B'nin altında C ve D vardır. Çağrılar şöyle ilerler (ara adım eklendi):

Çağrı	Yazılan	Sonra
A	A	sol: B
B	B	sol: C
C	C	çocuk yok, B'ye dönülür, B'nin sağı: D
D	D	çocuk yok, A'ya dönülür, A'nın sağı: E
E	E	çocuk yok, bitti

Yazılan sıra A, B, C, D, E'dir. Hocanın anlatımıyla A ile iş yaptınız, B ile iş yaptınız, varsa C'nin aşağısını, varsa D'nin aşağısını bitirdiniz, sol alt ağacın tamamı bitti, sağ alt ağaca, E'ye geçtiniz.

> **Ders sonrası not:** `printPreorder` `gcc -Wall` ile uyarısız derlendi. `A + B * C` ağacı elle kurulup üç gezinme de denendi; sonuçlar `+ A * B C`, `A + B * C` ve `A B C * +` çıktı. Yukarıdaki beş düğümlü ağaçta sıra A, B, C, D, E çıktı. Karakter yazdırmak için denemede `printf` biçimi `%c` yapıldı.

12.1 Öğrencilere Bırakılan Çalışma

Hoca `printInorder` ve `printPostorder` fonksiyonlarını yazmadı, öğrencilerin kendilerinin yazmasını istedi. Önerisi şöyledir:

1. Basit sayılarla ya da `A`, `B`, `C`, `+`, `*` gibi karakterlerle elle bir ifade ağacı kurun.
2. Önce preorder yazdırıp prefix gösterimini adım adım görün.
3. Sonra inorder ve postorder fonksiyonlarını kendi başınıza yazıp infix ve postfix gösterimlerini görün.

Soru: Ağacın oluşturulmasını da yazmamız gerekecek mi, yoksa elle mi oluşturacağız? **Cevap:** Bu örnek için elle oluşturun. Ağaca ekleme (insert) ve silmeyi (delete) ayrıca anlatacağım. Bu örnek yalnızca hangi sırayla gezildiğini görmek için. İsterseniz kendiniz de ekleme yazmayı deneyebilirsiniz; haftaya üzerinde oynamış olarak gelirseniz daha rahat edersiniz.

> **Hocanın vurgusu:** Ağaç ve graf en çok kullanılan veri yapılarıdır ve bağlı liste kullanırlar. Bu yüzden bağlı listeyi mutlaka bilmeniz gerekir. Gecikmeden üzerinde oynarsanız bilgiler kalıcı olur ve üzerine yenileri eklendikçe zorlanmazsınız.

Dersin Toparlaması

Bu derste kuyruk konusu tamamlandı ve ağaç konusuna girildi. Kuyruğun dizi ile iki gerçekleştirmesi hatırlatıldı: kaydırmalı yöntem her çıkışta $n - 1$'lik iş yapabilir, dairesel yöntem yalnızca işaretçileri hareket ettirir. Boyut kestirilemiyorsa bağlı liste kullanılır, bunun bedeli ek adres alanı ve bellek ayırma işleridir. Bağlı listeyle `createQueue`, `enQueue`, `newNode` ve `deQueue` yazıldı. En ince nokta, tek elemanlı kuyrukta `deQueue` sonrası `rear`'ın da `NULL` yapılmasıydı.

Ağaç tarafında kök, yaprak, iç düğüm, derece, seviye, derinlik, yükseklik, kenar, yol, alt ağaç ve orman terimleri tanımlandı. Ağacın dizi ile gösteriminin boş yer problemi, $2i + 1$ ve $2i + 2$ yerleşimi üzerinden görüldü; bu yüzden bağlı yapı tercih edilir. Preorder, inorder ve postorder gezinme ifade ağacında prefix, infix ve postfix gösterimlerini verir ve alt ağaç da bir ağaç olduğu için özyinelemeyle yazılır. Sonraki derste ağaca ekleme ve silme anlatılacak, ilerleyen haftalarda heap gibi başka ağaç türleri görülecektir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.